

DQL Query Language Guide

Searching Wazuh alerts with Dashboard Query Language

Security Lab

2026-07-06

Table of Contents

DQL Query Language Guide	1
1. What DQL is (and isn't)	1
2. Core syntax.....	2
3. Field types matter (text vs keyword)	3
4. Time filtering.....	4
5. Wazuh field reference (most-used).....	4
6. Query library (copy-paste)	5
7. DQL vs Lucene (when to switch).....	6
8. Saving & reusing queries	6
9. Practical tips	7

DQL Query Language Guide

Dashboard Query Language (DQL) for the Wazuh / OpenSearch Dashboards search bar

Applies to Wazuh dashboard search bar (Threat Hunting, Discover, any Explore view)

Language DQL — OpenSearch Dashboards Query Language (the default)

Audience SOC analysts writing ad-hoc searches and saved queries

Version 1.0 · 2026-07-06

Field names below use standard Wazuh alert fields. Substitute your own values for placeholders like <WINDOWS_HOSTNAME> and <METASPLOITABLE_IP>.

1. What DQL is (and isn't)

DQL is the query language in the search bar at the top of Wazuh's Threat Hunting / Discover views. You type a query, press Enter, and the event table + histogram filter to matches within the selected **time range** (top-right time picker).

- DQL is **field-aware** and forgiving: simple `field:value` syntax, boolean logic, wildcards, ranges, and existence checks.
- DQL is **not** Lucene. It does **not** support regular expressions or fuzzy matching. For those, toggle the language from **DQL** → **Lucene** (the “DQL” label at the left of the search bar is clickable) and use Lucene/`query_string` syntax.
- DQL and **filter pills** (the “+ Add filter” button) do the same job different ways — DQL is faster to type; filters are easier to toggle on/off and share. They combine with AND.

Golden rule: narrow by **time first** (the picker), then by **field**. A tight time window makes every query faster and every result easier to read.

2. Core syntax

2.1 Free-text vs field-scoped

Query	Meaning
<code>failed</code>	Free text — matches the term “failed” anywhere in the analyzed default fields
<code>rule.description:"authentication failed"</code>	Field-scoped phrase — that exact phrase in <code>rule.description</code>
<code>rule.description:failed</code>	Field-scoped term — the token “failed” in that field

Always prefer **field-scoped** queries in a SOC — they’re precise and fast. Free text is for quick “is it in here anywhere?” checks.

2.2 Quoting

- Use double quotes for **phrases** or values containing spaces/special characters:
`rule.description:"SSH insecure connection attempt".`
- Unquoted single tokens are fine: `agent.id:001`.

2.3 Boolean operators

and, or, not (case-insensitive). Combine and group with parentheses.

```
rule.level >= 10 and agent.name:"<WINDOWS_HOSTNAME>"
rule.groups:"authentication_failures" or rule.groups:"authentication_success"
not rule.groups:"authentication_success"
(rule.level >= 12) and (agent.name:"<WINDOWS_HOSTNAME>" or
data.srcip:"<METASPLOITABLE_IP>")
```

Precedence: not binds tightest, then and, then or. When mixing and/or, **use parentheses** to make intent explicit — don’t rely on precedence.

2.4 Comparison & ranges (numbers and dates)

```
rule.level > 7
rule.level >= 7 and rule.level <= 11      # medium band
data.virustotal.malicious > 0
```

Operators: >, >=, <, <=. For an exact number use field:value (rule.level:12).

2.5 Wildcards

* matches zero or more characters (there is no single-char ? in DQL).

```
agent.name:win*
data.win.eventdata.commandLine:*-enc*
syscheck.path:*CurrentVersion\\Run*
```

Leading wildcards (*foo) work but are slow on large indexes — anchor the front when you can.

2.6 Existence

field:* matches documents where the field **exists** (is present and non-null).

```
data.virustotal:*      # only alerts that went through VirusTotal
data.win.eventdata.commandLine:*
```

Negate for “field is missing”: not data.srcip:.*

2.7 Escaping special characters

These are special in DQL and must be escaped with a backslash when used literally: \ () : < > " *

- **Windows paths / backslashes:** a literal \ is written \\. Example: syscheck.path:"C:\\Windows\\System32\\drivers\\etc\\hosts". In practice it's easier to wildcard: syscheck.path:*System32*.
- **Colons in values** (e.g. IPv6): quote the value — data.srcip:"fe80::1".

3. Field types matter (text vs keyword)

OpenSearch stores two flavors of string fields, and it changes how matching behaves:

Type	Behavior	Typical Wazuh fields
keyword	Exact, case-sensitive, whole-value match	rule.id, rule.groups, agent.id, agent.name, data.srcip, location, decoder.name
text	Analyzed into tokens; matches individual words, case-insensitive	rule.description, full_log

Implications: - On **keyword** fields, `agent.name: "Win11-Host"` must match the whole value exactly. Use `*` for partial: `agent.name: Win*`. - On **text** fields, `rule.description: failed` matches the token “failed” inside a longer description. To match an exact phrase, quote it. - Some fields expose a `.keyword` sub-field for exact matching of an otherwise-text field (e.g. `rule.description.keyword: "..."`). If an exact match “won’t match”, try the `.keyword` variant, or switch to a wildcard.

4. Time filtering

- The **time picker** (top-right) is the primary time filter — Quick ranges (Last 15 min, 24 h, 7 days) or absolute start/end. Everything you type in DQL is evaluated **within** it.
- The time field for Wazuh alerts is `timestamp`. You can express relative math in filters, but for day-to-day work use the picker; it’s faster and less error-prone.
- **Auto-refresh** (next to the picker) is handy for live monitoring during an active incident.

5. Wazuh field reference (most-used)

Field	Meaning / example
<code>rule.id</code>	Rule that fired — <code>rule.id: "5710"</code>
<code>rule.level</code>	Severity 0–15 — <code>rule.level >= 12</code>
<code>rule.description</code>	Human description (text)
<code>rule.groups</code>	Categories — <code>authentication_failures, syscheck, virustotal, sshd, web</code>
<code>rule.mitre.id</code>	ATT&CK technique — <code>rule.mitre.id: "T1110"</code>
<code>rule.mitre.technique</code>	Technique name
<code>agent.id / agent.name</code>	Reporting agent — <code>agent.id: "001"</code>
<code>agent.ip</code>	Agent’s IP
<code>manager.name</code>	Manager node that processed the event
<code>location</code>	Log source/path or decoder origin (key for syslog sources)
<code>decoder.name</code>	Decoder that parsed it — <code>decoder.name: "sshd"</code>
<code>full_log</code>	The raw log line (text) — great for free-text pivots
<code>timestamp</code>	Event time (the time field)
<code>data.srcip</code>	Source IP (auth, network, syslog)
<code>data.srcuser / data.dstuser</code>	Source / target username
<code>data.srcport / data.dstport</code>	Ports
<code>data.win.system.eventID</code>	Windows Event ID — 4625, 4624, and Sysmon 1,3,10,11
<code>data.win.eventdata.image</code>	Sysmon process image path

data.win.eventdata.commandLine	Sysmon process command line
data.win.eventdata.parentImage	Sysmon parent process
data.win.eventdata.targetImage	Sysmon target (e.g. lsass.exe for EID 10)
syscheck.path	FIM changed file/registry path
syscheck.event	FIM event — added, modified, deleted
data.virustotal.malicious	VirusTotal malicious engine count
data.virustotal.source.file	File VT scanned
GeoLocation.country_name	Geo of data.srcip (if enabled)

Not sure of a field name? Expand any event in the table and click a field to add it as a filter, or use the **Available fields** panel on the left of Discover to browse and preview values.

6. Query library (copy-paste)

6.1 Severity & triage

```
rule.level >= 15 # critical
rule.level >= 12 # high + critical
rule.level >= 7 and rule.level <= 11 # medium band
rule.level >= 12 and agent.name:"<WINDOWS_HOSTNAME>"
```

6.2 Authentication

```
rule.groups:"authentication_failures"
rule.groups:"authentication_success"
data.win.system.eventID:"4625" # Windows failed logon
data.win.system.eventID:"4624" # Windows successful logon
data.srcuser:"administrator" and rule.groups:"authentication_failures"
```

6.3 Brute force / spraying

```
data.srcip:"<METASPLOITABLE_IP>" and rule.groups:"authentication_failures"
rule.mitre.id:"T1110" # Brute Force technique
```

6.4 Sysmon / process activity (Windows)

```
data.win.system.eventID:"1" # process create
data.win.eventdata.commandLine:"-enc*" # encoded PowerShell
data.win.eventdata.commandLine:>(*FromBase64String* or *IEX*)
data.win.eventdata.image:*powershell.exe* and
data.win.eventdata.parentImage:*winword.exe*
data.win.system.eventID:"3" # network connection
data.win.system.eventID:"10" and data.win.eventdata.targetImage:*lsass.exe*
# LSASS access
```

6.5 FIM / persistence

```
rule.groups:"syscheck"
syscheck.event:"added"
```

```
syscheck.path:*CurrentVersion\\Run* # autorun persistence
syscheck.path:*System32* and syscheck.event:"modified"
```

6.6 Malware / VirusTotal

```
rule.groups:"virustotal"
data.virustotal.malicious > 0
rule.groups:"virustotal" and data.virustotal.malicious >= 3
```

6.7 Metasploitable / syslog sources (agentless)

```
data.srcip:"<METASPLOITABLE_IP>"
location:*syslog* and rule.groups:"sshd"
full_log:*"<METASPLOITABLE_IP>"* # free-text pivot on the
raw log
```

(See the dedicated **Searching Metasploitable Events** guide for the full approach.)

6.8 MITRE-driven hunts

```
rule.mitre.id:"T1059" # Command & Scripting
Interpreter
rule.mitre.id:"T1003" # OS Credential Dumping
rule.mitre.technique:*PowerShell*
```

6.9 Exclusions / noise reduction

```
rule.level >= 7 and not rule.groups:"authentication_success"
rule.groups:"syscheck" and not syscheck.path:*\\Temp\\*
```

7. DQL vs Lucene (when to switch)

Toggle the language via the **DQL** label at the left of the search bar.

Need	Use
Everyday field/boolean/wildcard/range	DQL (default)
Regular expressions	Lucene — rule.description:/auth.*fail/
Fuzzy matching (typos)	Lucene — rule.description:authetication~
Boosting / advanced scoring	Lucene

Most SOC work stays in DQL; reach for Lucene only when you need regex or fuzzy.

8. Saving & reusing queries

- **Save a search:** once a query + columns are set in Discover, use **Save** to store it; reopen it from **Open**. Great for recurring hunts (e.g. "Metasploitable SSH failures").
- **Filter pills** can be **pinned** to persist across views, **negated** (is not), toggled, and edited without retyping.

- **Dashboards / visualizations:** turn a saved search into a panel (bar chart of `rule.level`, data table by `data.srcip`, etc.) for at-a-glance monitoring.
-

9. Practical tips

- **Start broad, then subtract.** Begin with a level/time filter, read a few events, then add field constraints to remove noise.
- **Pivot from an event.** Expand any alert and click a field value → *Filter for* / *Filter out* to build queries without memorizing field names.
- **Case sensitivity:** keyword fields are case-sensitive; if an exact match fails, try a wildcard or the `.keyword` variant, or check the actual stored value in an expanded event.
- **Quote anything with spaces, colons, or slashes.**
- **When a query returns nothing, relax it** one clause at a time to find which term is too strict.
- **Save your best hunts** so triage becomes one click next shift.

End of guide.